



Projet parrallélisme

Avec l'utilisation d'OpenMP et OpenMPI

Auteurs :

BELKADI Farès
CHENNOUF Ziyad
GAYON Mathis
NADAUD Antonin

Encadrant :

GARCIA Thierry

10 juin 2026

Sommaire

1	Introduction	3
2	Compréhension séquentielle	3
3	Compréhension du code séquentiel	4
3.1	Compilation et exécution	4
3.2	Boucle principale de calcul	4
3.3	Données lues et écrites	4
3.4	Potentiel de parallélisation	4
3.5	Test	5
3.6	Détails techniques	5
4	Parallélisation avec OpenMP	6
4.1	Modifications du code	6
4.2	Temps d'exécution et speedup	7
5	Parallélisation avec Open MPI	8
5.1	Principe de MPI	8
5.2	Initialisation des processus	8
5.3	Découpage horizontal de la grille	9
5.4	Mémoire locale et lignes fantômes	9
5.5	Placement des premiers individus infectés	10
5.6	Échange des frontières	10
5.7	Calcul local de la propagation	11
5.8	Obtention du résultat global	11
5.9	Compilation et exécution	11
6	Analyse des résultats	12
7	Conclusion	13
8	Bibliographie	14

1 Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

$$E = mc^2 \tag{1}$$

2 Compréhension séquentielle

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

3 Compréhension du code séquentiel

3.1 Compilation et exécution

Le code séquentiel fourni a été compilé et exécuté avec succès. Cette exécution fournit le temps d'exécution de référence, nécessaire pour mesurer le gain de performance (*speedup*) des versions parallèles.

3.2 Boucle principale de calcul

La boucle temporelle principale est `for (int it = 0; it < ITER; ++it)`. À l'intérieur se trouve une double boucle spatiale (indices `i` et `j`, de 1 à $N-1$) qui met à jour les cellules de la grille à chaque itération.

3.3 Données lues et écrites

Durant la phase de calcul :

- **Lues** : l'état courant `A[IDX(i,j)]` et l'état des quatre voisins (haut, bas, gauche, droite).
- **Écrites** : la nouvelle valeur de la cellule dans `B[IDX(i,j)]`.

Note : à la fin de chaque pas temporel, les rôles/pointeurs de `A` et `B` sont échangés.

3.4 Potentiel de parallélisation

La mise à jour de `B[IDX(i,j)]` dépend uniquement des valeurs lues dans `A` (lecture seule pendant l'itération spatiale). Il n'y a donc pas de dépendance entre deux itérations distinctes sur `i,j`. Les boucles spatiales peuvent donc être parallélisées via OpenMP (`#pragma omp parallel for`) ou distribuées par domaines avec MPI.

```
1 def factorial(n):
2     if n == 0:
3         return 1
4     else:
5         return n * factorial(n-1)
6
7 print(factorial(5))
```

3.5 Test

Ceci est une boîte avec un fond gris clair et un titre qui reprend la numérotation de la section.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Test aa Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

```
1 def factorial(n):
2     if n == 0:
3         return 1
4     else:
5         return n * factorial(n-1)
6
7 print(factorial(5))
```

3.6 Détails techniques

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec no-



nummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

4 Parallélisation avec OpenMP

4.1 Modifications du code

Afin de réaliser notre parallélisation, nous utilisons les directives de la bibliothèque **omp.h**. Nous avons identifié les boucles parallélisables de notre code en amont afin d'éviter des résultats incohérents. Voici ci-dessous les parties du code ayant été modifiées :

```
1 // Boucle de remplissage des tableaux
2 #pragma omp parallel for
3 for(int i=0;i<N;i++) {
4     for(int j=0;j<N;j++) {
5         A[IDX(i,j)] = 0;
6         B[IDX(i,j)] = 0;
7     }
8 }
```

```
1 // Boucle principale de traitement
2 // On boucle ITER fois
3 for(int it=0; it<ITER; it++) {
4     // On boucle sur une dimension du tableau ie N fois
5     #pragma omp parallel for
6     for(int i=1;i<N-1;i++) {
7
8         ...
9
10    // On inverse A et B pour que A soit le tableau final
11    int *tmp = A;
```

```

12     A = B;
13     B = tmp;
14 }

```

```

1  // Première dimension du comptage des individus
2  #pragma omp parallel for reduction(+:c0,c1,c2)
3  for(int i=0;i<N;i++) {
4
5      // Deuxième dimension du parcours
6      for(int j=0;j<N;j++) {
7          if(A[IDX(i,j)] == 0) c0++;
8          else if(A[IDX(i,j)] == 1) c1++;
9          else c2++;
10     }
11 }

```

La boucle du premier cadre permet seulement de remplir le tableau utilisé pour représenter la population étudiée. On peut paralléliser cette opération car l'ordre de remplissage du tableau n'importe pas.

La boucle du second cadre correspond à l'opération de propagation de la maladie. La parallélisation est rendue possible par l'utilisation de deux tableaux différents. Le tableau A sert à la lecture des valeurs tandis que le tableau B sert à l'écriture des valeurs. Ainsi, il n'y a aucun conflit dans l'ordre de mise à jour des valeurs.

Enfin, le dernier cadre agit sur le calcul du nombre de personnes saines, infectées et guéries. On utilise une réduction pour que les threads somment les valeurs à la fin de la parallélisation. En effet, sans la directive de réduction, on observe des individus manquants dans le résultat.

4.2 Temps d'exécution et speedup

Dans un premier temps, nous avons réalisé des mesures sur une population relativement faible de 4 000 000 de personnes. Cela nous permet d'avoir une idée de l'efficacité de la parallélisation sur de petites populations. Le nombre d'itérations est fixé à 500, on pourrait donc dire que l'on étudie l'épidémie sur 500 jours.

Threads	Temps d'exécution	Speedup
1	9,333s	1
2	4,998s	1,867
4	6,076s	1,536
8	3,003s	3,108

Dans un second temps, nous avons réalisé notre simulation sur une population environ égale à celle de la France (69 089 340). Ainsi, nous voyons quel temps une simulation prendrait à l'échelle du pays, bien que le modèle de propagation soit très simple.

Threads	Temps d'exécution	Speedup
1	2min 33s	1
2	1min 30s	1,7
4	1min 17s	1,99
8	48s	3,19

5 Parallélisation avec Open MPI

5.1 Principe de MPI

MPI (*Message Passing Interface*) est une bibliothèque permettant de programmer des applications parallèles par échange de messages. Open MPI est une implémentation libre de cette bibliothèque. Contrairement aux threads OpenMP, les processus MPI ne partagent pas leurs variables : chaque processus possède sa propre mémoire locale.

Le même programme `epidemie_mpi.c` est exécuté par tous les processus. Chaque processus travaille cependant sur une partie différente de la grille, déterminée grâce à son numéro.

5.2 Initialisation des processus

Les premières primitives MPI utilisées dans le programme sont les quatre fonctions essentielles présentées dans le cours :

```

1 MPI_Init(&argc, &argv);
2 MPI_Comm_rank(MPI_COMM_WORLD, &rang);
3 MPI_Comm_size(MPI_COMM_WORLD, &nbproc);

```

`MPI_Init` démarre l'environnement MPI. `MPI_Comm_rank` place dans `rang` le numéro unique du processus courant et `MPI_Comm_size` place dans `nbproc` le nombre total de processus. Le communicateur `MPI_COMM_WORLD` contient ici tous les processus lancés. Les rangs sont numérotés de 0 à `nbproc - 1`.

À la fin du programme, chaque processus appelle `MPI_Finalize` pour terminer l'environnement MPI.

5.3 Découpage horizontal de la grille

Dans la version séquentielle, un seul programme possède les $N \times N$ cellules. Dans la version MPI, nous avons choisi un découpage par blocs de lignes. Chaque processus reçoit le même nombre de lignes :

```
1 int bloc = N / nbproc;
2 int debut = rang * bloc;
```

`bloc` est le nombre de lignes traitées localement et `debut` est l'indice global de la première ligne du bloc. Le programme vérifie que N est divisible par le nombre de processus afin que tous les blocs aient la même taille.

Pour $N = 300$ et quatre processus, la répartition est la suivante :

Processus	Lignes globales traitées
0	0 à 74
1	75 à 149
2	150 à 224
3	225 à 299

Ce choix suit le principe du découpage par blocs vu en cours : chaque processus effectue indépendamment un calcul local sur une partie des données.

5.4 Mémoire locale et lignes fantômes

Pour déterminer le nouvel état d'une cellule, le programme lit ses quatre voisins orthogonaux. Les voisins de gauche et de droite appartiennent toujours au même bloc. En revanche, le voisin du haut de la première ligne locale et le voisin du bas de la dernière ligne locale peuvent appartenir à un autre processus.

Chaque processus alloue donc ses `bloc` lignes, ainsi que deux lignes supplémentaires :

```
1 int *A = malloc((size_t)(bloc + 2) * N * sizeof(int));
2 int *B = malloc((size_t)(bloc + 2) * N * sizeof(int));
```

-
- la ligne locale 0 reçoit la frontière du processus précédent ;
 - les lignes 1 à `bloc` sont réellement calculées ;
 - la ligne `bloc + 1` reçoit la frontière du processus suivant.

Ces deux lignes supplémentaires sont appelées lignes fantômes. Elles contiennent une copie des données reçues et permettent d'utiliser la même règle de propagation que dans la version séquentielle.

5.5 Placement des premiers individus infectés

Les coordonnées des trois premiers individus infectés sont globales. Chaque processus vérifie donc si la ligne correspondante appartient à son bloc. Seul le processus propriétaire modifie sa grille locale :

```

1 if (lignes_infectees[k] >= debut &&
2     lignes_infectees[k] < debut + bloc) {
3     int ligne_locale = lignes_infectees[k] - debut + 1;
4     A[IDX(ligne_locale, colonnes_infectees[k])] = 1;
5 }
```

Le décalage de 1 correspond à la ligne fantôme placée au début du tableau.

5.6 Échange des frontières

Avant chaque itération, les processus voisins doivent mettre à jour leurs lignes fantômes. Le programme utilise les communications point à point `MPI_Send` et `MPI_Recv` présentées dans le cours.

```

1 MPI_Send(&A[IDX(bloc,0)], N, MPI_INT, rang + 1, 0,
2         MPI_COMM_WORLD);
3 MPI_Recv(&A[IDX(bloc + 1,0)], N, MPI_INT, rang + 1, 1,
4         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Dans cet exemple, le processus envoie N entiers de sa dernière ligne réelle au processus suivant. Il reçoit ensuite N entiers dans sa ligne fantôme du bas. Les paramètres indiquent la donnée, le nombre d'éléments, leur type, le rang du processus distant, l'étiquette du message et le communicateur.

Les échanges sont organisés en deux étapes :

1. communications entre les processus 0 et 1, 2 et 3, etc. ;
2. communications entre les processus 1 et 2, 3 et 4, etc.

Dans chaque paire, un processus envoie pendant que l'autre reçoit. Cet ordre est important car `MPI_Recv` attend l'arrivée de la donnée.

5.7 Calcul local de la propagation

Une fois les frontières reçues, tous les processus calculent en parallèle leurs propres lignes. La règle de propagation reste identique : une personne saine devient infectée si au moins deux de ses quatre voisins sont infectés, une personne infectée devient guérie et une personne guérie reste guérie.

```
1 for (int i = 1; i <= bloc; i++) {
2     int ligne_globale = debut + i - 1;
3
4     if (ligne_globale != 0 && ligne_globale != N - 1) {
5         for (int j = 1; j < N - 1; j++) {
6             /* Meme calcul que dans la version
7                sequentielle */
8         }
9     }
```

La variable `ligne_globale` permet de reconnaître les bords extérieurs de la grille. Comme dans la version séquentielle, le tableau `A` est lu et le nouvel état est écrit dans `B`. Les deux pointeurs sont ensuite échangés.

5.8 Obtention du résultat global

À la fin de la simulation, chaque processus compte les individus sains, infectés et guéris de son bloc. Ces compteurs locaux sont regroupés avec une communication collective :

```
1 MPI_Reduce(compteurs_locaux, compteurs_globaux, 3,
2           MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

`MPI_Reduce` additionne les trois compteurs de tous les processus grâce à l'opération `MPI_SUM`. Le résultat est placé dans `compteurs_globaux` sur le processus de rang 0. Une seconde réduction additionne les temps locaux. Seul le processus 0 affiche les résultats finaux.

5.9 Compilation et exécution

Le programme est compilé avec `mpicc`, qui ajoute automatiquement les bibliothèques MPI nécessaires :

```
mpicc epidemie_mpi.c -o epidemie_mpi
mpirun -np 4 ./epidemie_mpi
```

L'option `-np 4` lance ici quatre processus. Ils exécutent tous le même programme, mais chacun possède son espace mémoire et calcule son bloc de lignes. Le nombre choisi doit diviser N avec le découpage actuel.

6 Analyse des résultats

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

7 Conclusion

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

8 Bibliographie

Voici la liste:

- [INSEE](#)
- [Google](#)
- [Wikipedia](#)